

PLATAFORMAS BIG DATA

3º de Ingeniería Informática

Práctica 1

El almacenamiento de los datos

Contexto

En la actualidad, el crecimiento de la inteligencia artificial (IA), el análisis masivo de datos y los servicios en la nube han incrementado de forma exponencial la necesidad de procesar y almacenar grandes volúmenes de información. Empresas como Google, Amazon, Microsoft o Meta disponen de centros de procesamiento de datos (CPD) formados por miles de servidores que ejecutan aplicaciones y almacenan la información de millones de usuarios de forma continua.

Estos CPD no solo prestan servicios como motores de búsqueda, comercio electrónico, asistentes inteligentes o plataformas de almacenamiento en la nube, sino que también se utilizan para ejecutar aplicaciones científicas y de ingeniería de gran complejidad. Entre ellas destacan el entrenamiento de modelos de inteligencia artificial, el procesamiento de grandes bases de datos, la resolución de problemas de álgebra lineal, las simulaciones numéricas, los estudios genómicos o la predicción meteorológica, entre muchas otras.

En todas estas aplicaciones, el rendimiento computacional es un factor determinante. Sin embargo, antes de optimizar un algoritmo o distribuir el trabajo entre múltiples servidores, es imprescindible comprender cómo se almacenan los datos en memoria. Una organización adecuada de la información permite reducir el consumo de memoria, mejorar la localidad espacial y temporal de los datos, disminuir el número de fallos de caché y aumentar el rendimiento del procesador durante su acceso.

Objetivos

Al finalizar la práctica el alumno será capaz de:

- La importancia de ciertas operaciones de álgebra lineal en el panorama actual de la Inteligencia Artificial y las plataformas Big Data.
- Comprender cómo se almacena de forma eficiente los datos en memoria.
- Analizar la organización contigua de datos en memoria.
- Entender el concepto de localidad espacial.

EJERCICIO 1

En primer lugar, es de vital importancia comprender la base de la IA, y qué operaciones fundamentales se llevan a cabo en los modelos de redes neuronales más usados en la actualidad.

Existen multitud de modelos de redes neuronales:

- **Redes neuronales convolucionales:** utilizadas por la comunidad para la predicción de eventos, estadísticas, detección de objetos, segmentación, etc.
- **Transformers encoders y decoders:** redes neuronales que tuvieron un gran impacto en la sociedad al poder llevar a cabo conversaciones con los usuarios de todo el mundo y poder llevar a cabo tareas complejas a nivel de desarrollo de software, diseño gráfico, traducciones de texto, etc.
- **Redes neuronales basadas en grafos:** otro tipo de redes neuronales basadas en grafos destinadas al estudio de estadísticas y relaciones de datos (citas en artículos científicos, estadísticas de compras en Amazon, relaciones de amistades en Facebook y otras redes sociales, etc.).

Existe un número mucho mayor de modelos de redes neuronales, además, de multitud de variantes que cada año se van multiplicando y haciendo estos modelos más eficientes y también más complejos.

Cada modelo de red neuronal contiene sus propias operaciones, pero todos ellos parten de una característica común:

LA OPERACIÓN MÁS COSTOSA ES EL PRODUCTO MATRICIAL (GEMM)

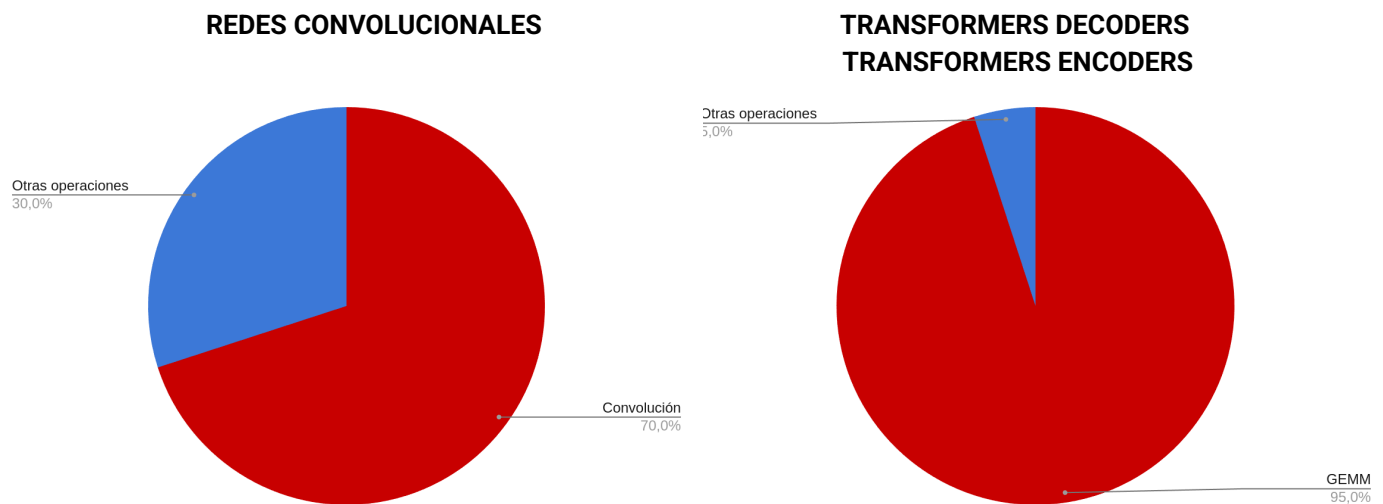


Figura 1. Coste del producto matricial y convolucional en diferentes tipos de modelos de IA.

La imagen de la izquierda muestra que alrededor del 65% del tiempo de cómputo al realizar una red convolucional se consume en la operación de convolución. Por motivos de paralelismo y eficiencia de rendimiento, es común que la mayoría de las bibliotecas de álgebra lineal destinadas al procesamiento de este tipo de operaciones (BLIS, OpenBLAS, MKL, XNNPACK, ARMCL, ARMPL, etc.) usen el método im2col/im2row para convertir la operación de convolución en una GEMM.

La imagen de la derecha muestra cómo, en el proceso de inferencia de Transformers encoders o Transformers decoders (proceso que se lleva a cabo millones y millones de veces por segundo en la actualidad, ya que cada consulta a plataformas como ChatGPT, Claude, Gemini, etc. se realiza mediante este tipo de inferencia), la operación más costosa con diferencia es el producto matricial.

Dicho esto, nuestro objetivo va a ser trabajar en acelerar este tipo de operaciones de álgebra lineal al máximo.

Para comenzar, es de vital importancia disponer de los datos almacenados de forma adecuada. Para el primer ejercicio, el alumno debe conocer los dos principales tipos de almacenamiento de matrices en memoria: **row-major** y **column-major**.

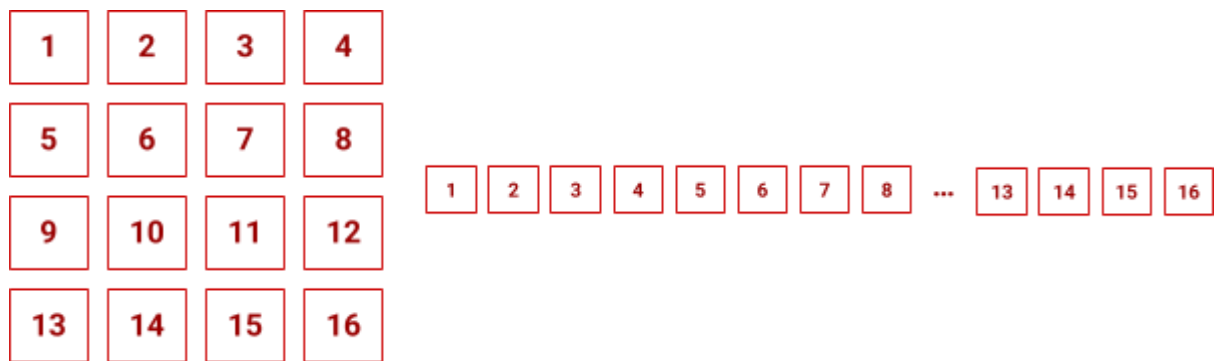


Figura 2. Almacenamiento de matriz basado en la estrategia row-major.

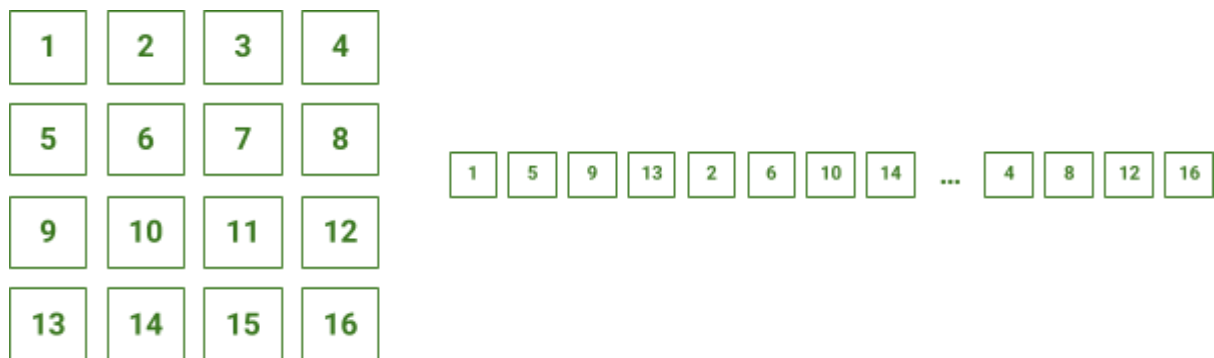


Figura 3. Almacenamiento de matriz basado en la estrategia column-major.

Con base en esto, el alumno deberá desarrollar en lenguaje de programación C/C++ un programa que cree 1 matriz de enteros M de tamaño $M=N=K=4$ (inicializada con valores aleatorios entre 0 y 10) . La reserva de la matriz se llevará a cabo siguiendo el almacenamiento estándar conocido por el alumnado (doble puntero `[][]` o `**`). Además, se considerará que esta matriz original está almacenada por filas, es decir, siguiendo la estrategia row-major.

EJERCICIO 2

A continuación, se deberán implementar 2 rutinas:

```
void *row_major_transform(DTYPE **orig, DTYPE *dest,  
                        const size_t M, const size_t N);
```

```
void *row_major_transform(DTYPE **orig, DTYPE *dest, const size_t M,  
                        const size_t N);
```

Cada una de ellas recibirá un doble puntero a la matriz original y deberá transformar este formato de almacenamiento en memoria al formato que corresponda y en el buffer de destino pasado como parámetro. Para ello, recorrerá la matriz original por filas (almacenamiento que usamos por defecto) y se almacenará en el nuevo formato correspondiente y de manera serializada.

Para ellos, el alumno inicialmente llevará a cabo 2 reservas de memoria siguiendo el estándar de C (malloc) y con un único puntero (*). Seguidamente, se implementarán las rutinas y se llamarán pasando como parámetros los buffers creados.

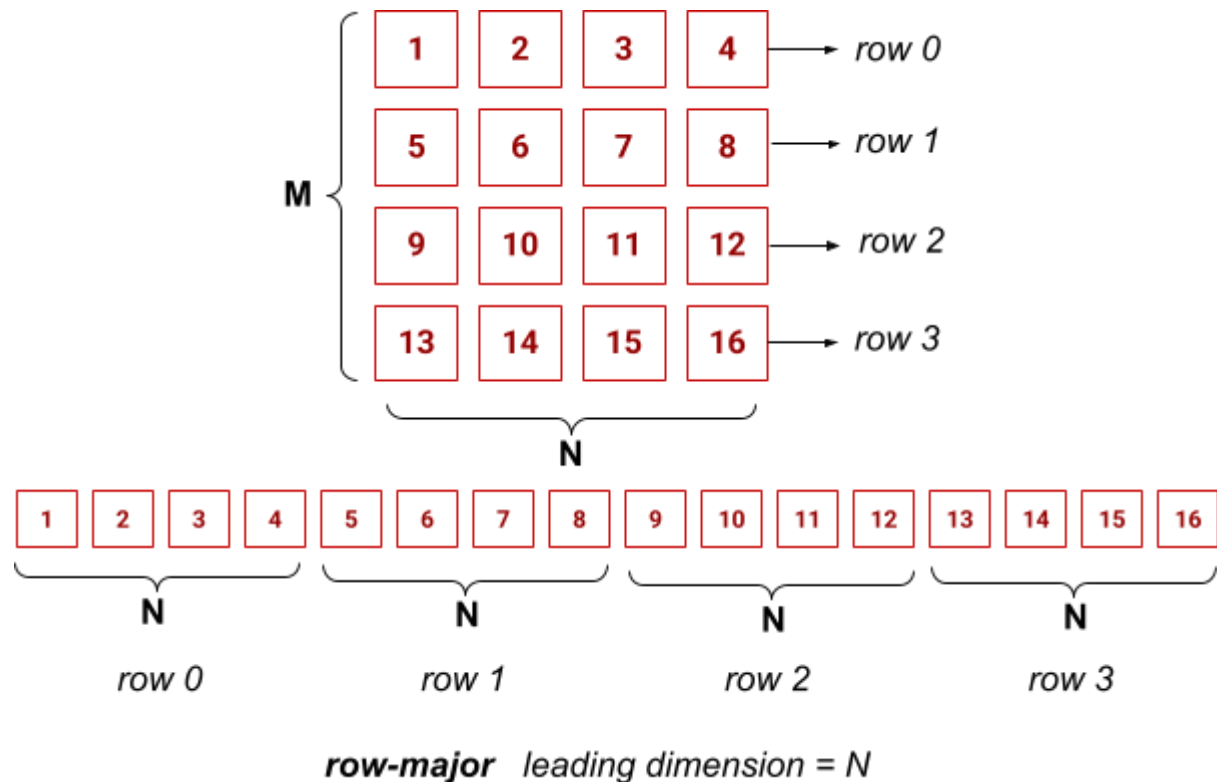


Figura 4. Esquema de almacenamiento de una matriz $M \times N$ en formato *row-major*.

EJERCICIO 3

Para comprobar el orden en el que se han almacenado los datos, en primer lugar, el alumno deberá mostrar por pantalla los valores que contienen cada una de las posiciones de los dos buffers rellenos por las rutinas anteriormente implementadas. Por pantalla deberá aparecer una secuencia de valores similares a las que aparecen en la parte derecha de la Figura 2 y la Figura 3.

EJERCICIO 4

Seguidamente, el alumno mostrará por pantalla la matriz original, con formato similar al que aparece a la parte izquierda en las Figuras 2 y 3, así como las matrices almacenadas en cada uno de los buffers. En este caso, nótese que la matriz es la misma en los 3 escenarios y la diferencia es el tipo de almacenamiento, por este motivo, la salida por pantalla debe ser similar en los tres escenarios.

EJERCICIO 5

El alumno deberá mostrar por pantalla las direcciones de almacenamiento de cada uno de los elementos almacenados en las tres matrices generadas y reservadas en memoria y analizar el direccionamiento.